

DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	V.VISRUTH
ROLL NO	AV.SC.U4CSE25250
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	DR GAYATHRI PARASA
DESIGNATION	ASST. PROFESSOR (SR. GR)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID 's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	5-8
2.	**WEEK 2:** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	9-11
3.	**WEEK-3:** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	12-13
4.	**WEEK-4:** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	14-16
5.	**WEEK-5:** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value	17-22

	6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10) Search data in SLL 11) Display or Traverse 12) Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10) Exit	23-29
7.	**WEEK-7:** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	30-37
8.	**WEEK-8:** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	38-51
9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals – in-order, pre-order and post-order	52-59

	4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	60-66
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	67-73
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	74-82

WEEK 1:

AIM:

1(A). A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by “Launch!”.

CODE:

rocket_Launch_system.py - C:\Users\visr\Desktop\DSA Lab\rocket_Launch_system.py (3.13.7)

File Edit Format Run Options Window Help

```
def countdown(n):
    if n <= 0:
        print("Launch")
    else:
        print(n)
        countdown(n - 1)
n = int(input("Enter n: "))
countdown(n)
```

OUTPUT:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/rocket_Launch_system.py ===
Enter n: 10
10
9
8
7
6
5
4
3
2
1
Launch
>>>
```

1(B). Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years.

CODE:

Power_Recursion.py - C:\Users\visr\Desktop\DSA Lab\Power_Recursion.py (3.13.7)

File Edit Format Run Options Window Help

```
p = float(input("Enter the Principal Growth Factor:"))
n = int(input("Enter the Number of Years:"))

def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

Output = power(p, n)
print("Growth of an Investment", Output)
```

OUTPUT:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: C:\Users\manee\OneDrive\Desktop\DSA Lab\Power_Recursion.py =====
Enter the Principal Growth Factor:2
Enter the Number of Years:3
Growth of an Investment 8.0
>>>
```

1(C). A HR department stores employee ID ’s in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

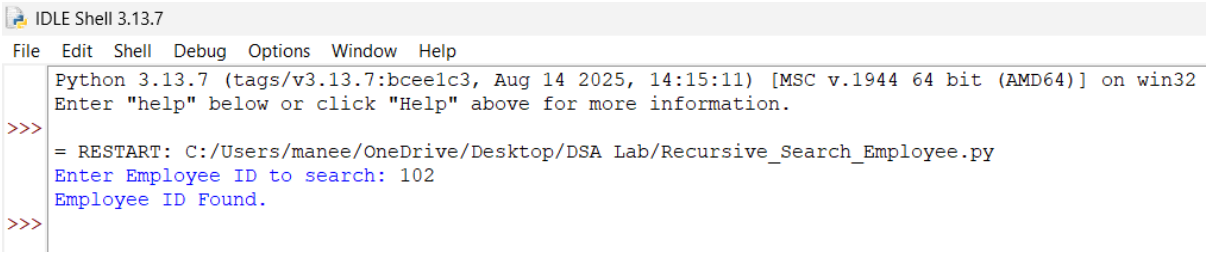
CODE:

Recursive_Search_Employee.py - C:\Users\visr\Desktop\DSA Lab\Recursive_ Search Employee.py (3.13.7)

File Edit Format Run Options Window Help

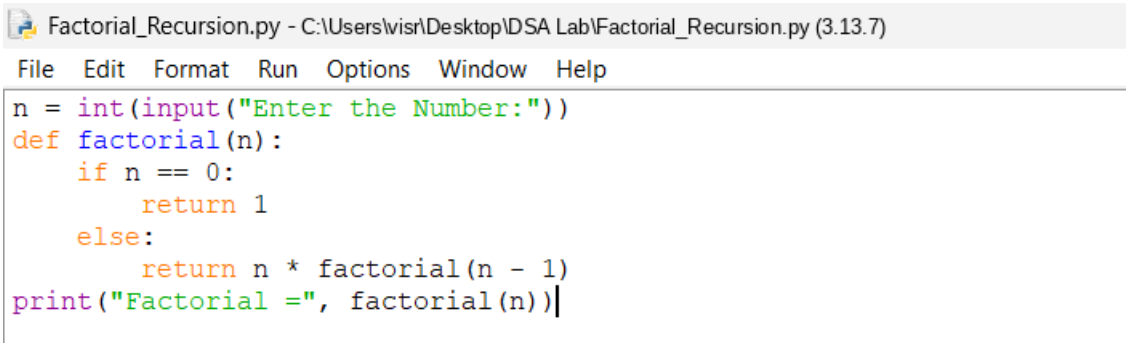
```
def employee(emp_list, target, index=0):
    if index == len(emp_list):
        return False
    if emp_list[index] == target:
        return True
    return employee(emp_list, target, index + 1)
employees = [101, 102, 103, 104, 105]
target_id = int(input("Enter Employee ID to search: "))
if employee(employees, target_id):
    print("Employee ID Found.")
else:
    print("Employee ID Not Found.")
```

OUTPUT:

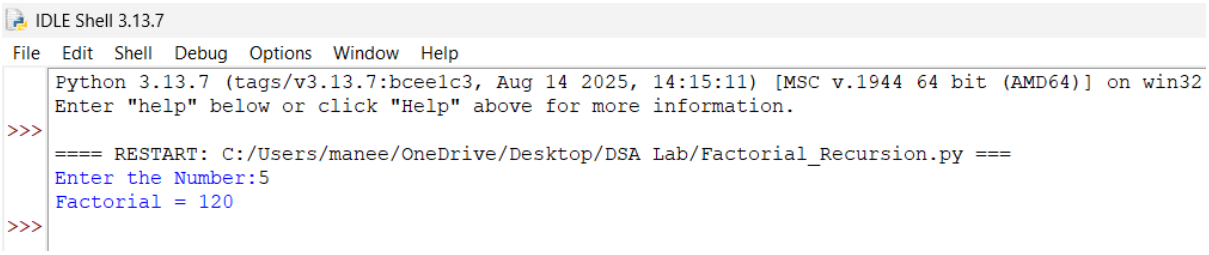


1(D). In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

CODE:



OUTPUT:



1(E). Write a recursive python program to print the first n terms of the Fibonacci series.

CODE:

Fibonacci_Recursion.py - C:\Users\visr\Desktop\DSA Lab\Fibonacci_Recursion.py (3.13.7)

File Edit Format Run Options Window Help

```
n = int(input("Enter number of terms:"))
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

for i in range(n):
    print(fibonacci(i), end=" ")
```

OUTPUT:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Fibonacci_Recursion.py ===
Enter number of terms:7
0 1 1 2 3 5 8
>>>
```

WEEK 2:

AIM:

2(A). Write a python program to implement linear search.

CODE:

Linear_Search.py - C:\Users\visr\Desktop\DSA Lab\Linear_Search.py (3.13.7)

File Edit Format Run Options Window Help

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

n = int(input("Enter the number of elements:"))
arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

key = int(input("Enter the element to search:"))

result = linear_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

OUTPUT:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
======= RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Linear_Search.py ======
Enter the number of elements:5
Enter the elements:
10
20
30
40
50
Enter the element to search:30
Element found at index 2
>>>
```

2(B). Write a python program to implement binary search with and without sorted array as an input.

CODE:

Without Sorted Array:

Binary_Search_Without_Sorted_Array.py - C:\Users\visr\Desktop\DSA Lab\Binary Search Without Sorted_Array.py (3.13.7)

File Edit Format Run Options Window Help

```
def binary_search(arr, key):
    arr.sort()
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1

arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter element to search: "))

result = binary_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

OUTPUT:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] c
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Binary_Search_Without_Sorted_Array.py
Enter elements: 40 10 50 20 30
Enter element to search: 30
Element found at index 2
>>>
```

CODE:

With Sorted Array:

Binary_Search_With_Sorted_Array.py - C:\Users\visr\Desktop\DSA Lab\Binary Search With Sorted_Array.py (3.13.7)

File Edit Format Run Options Window Help

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1

arr = list(map(int, input("Enter sorted elements: ").split()))
key = int(input("Enter element to search: "))

result = binary_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

OUTPUT:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Binary_Search_With_Sorted_Array.py
Enter sorted elements: 10 20 30 40 50
Enter element to search: 40
Element found at index 3
>>>
```

WEEK-3:

AIM:

3(A). Write a python program to implement bubble sort.

CODE:

Bubble_Sort.py - C:\Users\visr\Desktop\DSA Lab\Bubble Sort. py (3.13.7)

File Edit Format Run Options Window Help

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
arr = list(map(int, input("Enter elements: ").split()))
result = bubble_sort(arr)
print("Sorted array:", result)
```

OUTPUT:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 6
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Bubble_Sort.py
Enter elements: 50 20 40 10 30
Sorted array: [10, 20, 30, 40, 50]
>>>
```

3(B). Write a python program to implement insertion sort.

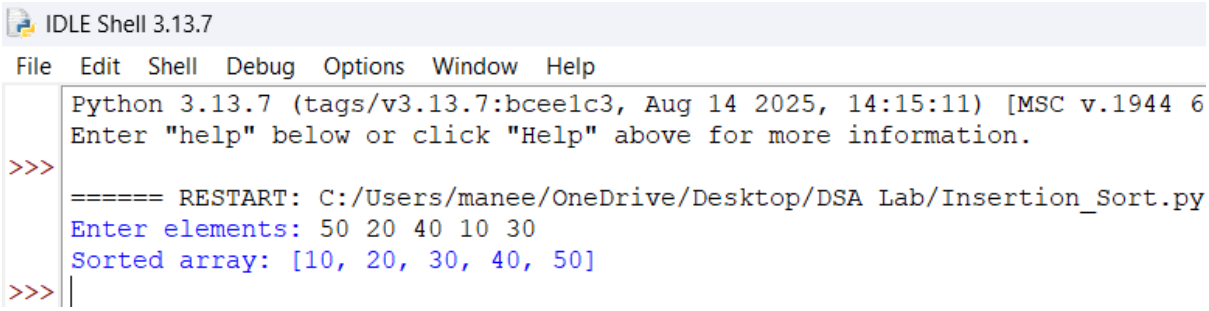
CODE:

Insertion_Sort.py - C:\Users\visr\Desktop\DSA Lab\Insertion_Sortpy (3.13.7)

File Edit Format Run Options Window Help

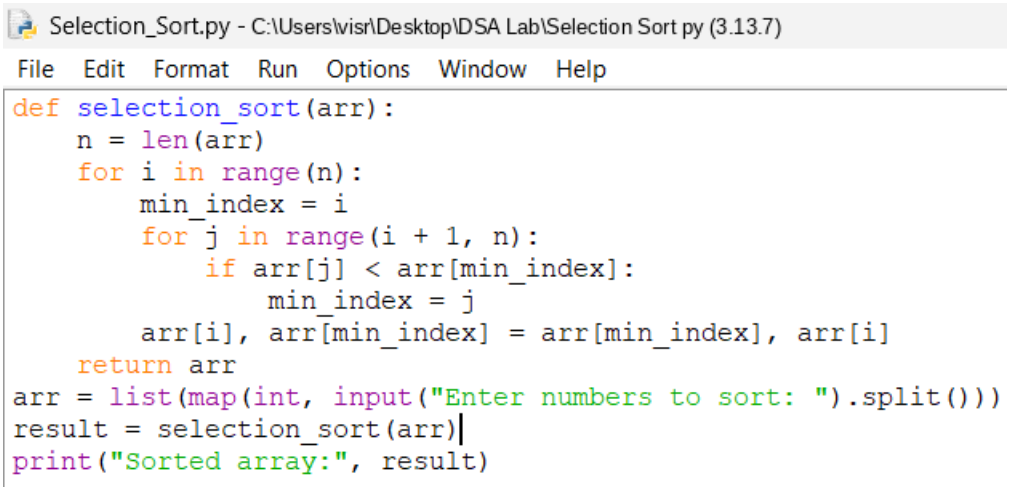
```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
arr = list(map(int, input("Enter elements: ").split()))
result = insertion_sort(arr)
print("Sorted array:", result)
```

OUTPUT:

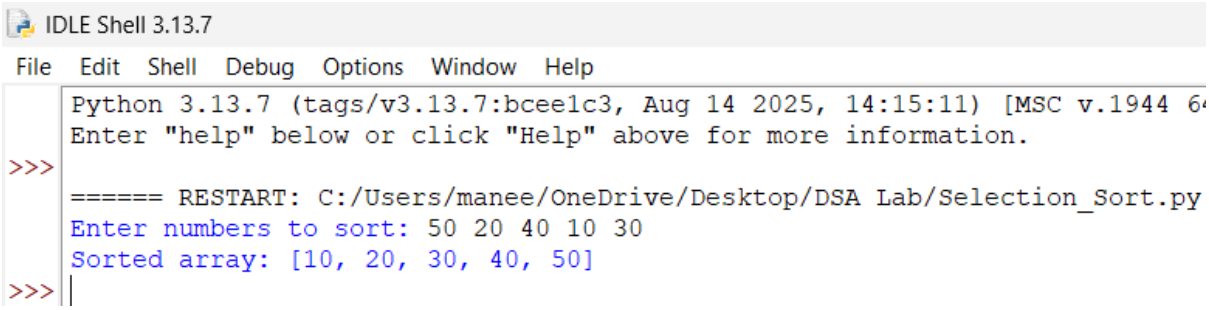


3(C). Write a python program to implement selection sort.

CODE:



OUTPUT:



WEEK-4:

AIM:

(A). Write a python program to implement quick sort.

CODE:

Quick_sort.py - C:\Users\visr\Desktop\DSA Lab\Quick_sort.py (3.13.7)

File Edit Format Run Options Window Help

```
def quicksort(a, low, high):
    if low < high:
        i = low
        j = high
        pivot = low

        while i < j:
            while i < len(a) and a[i] <= a[pivot]:
                i += 1

            while a[j] > a[pivot]:
                j -= 1

            if i < j:
                a[i], a[j] = a[j], a[i]

        a[j], a[pivot] = a[pivot], a[j]

        quicksort(a, low, j - 1)
        quicksort(a, j + 1, high)


a = list(map(int, input("Enter numbers to sort: ").split(" ")))
n = len(a)
quicksort(a, 0, n - 1)

print("Sorted array:")
print(a)
```

OUTPUT:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Quick_sort.py ========
Enter numbers to sort: 30 10 20
Sorted array:
[10, 20, 30]
>>>
```

(B). Write a python program to implement merge sort.

CODE:

Merge_Sort.py - C:\Users\visr\Desktop\DSA Lab\Merge_Sort.py (3.13.7)

File Edit Format Run Options Window Help

```
def merging(arr, low, mid, high):
    result = []
    i = low
    j = mid + 1

    while i <= mid and j <= high:
        if arr[i] < arr[j]:
            result.append(arr[i])
            i += 1
        else:
            result.append(arr[j])
            j += 1

    while i <= mid:
        result.append(arr[i])
        i += 1

    while j <= high:
        result.append(arr[j])
        j += 1

    i = low
    j = 0

    while i <= high:
        arr[i] = result[j]
        i += 1
        j += 1


def merge_sort(arr, low, high):
    if low < high:
        mid = (low + high) // 2
        merge_sort(arr, low, mid)
        merge_sort(arr, mid + 1, high)
        merging(arr, low, mid, high)


arr = list(map(int, input("Enter numbers to sort: ").split()))
n = len(arr)
merge_sort(arr, 0, n - 1)

print("Sorted array:", arr)
```

OUTPUT:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Merge_Sort.py ========
Enter numbers to sort: 30 10 20
Sorted array: [10, 20, 30]
>>>
```

WEEK – 5

AIM:

(A): Write a python program to implement a Singly Linked List with the below operations.

1) Create Linked List
2) Insert Beginning
3) Insert End
4) Insert at Index
5) Delete by Value
6) Delete First Node
7) Delete Last Node
8) Count number of nodes
9) Sort the data of SLL
10) Search data in SLL
11) Display or Traverse
12) Exit

CODE:

SLL.py - C:\Users\visr\Desktop\DSA Lab\SLLpy (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data=data
        self.next=None

class SinglyLinkedListList:
    def __init__(self):
        self.head=None

    def create(self):
        n=int(input(("Enter number of elements:")))
        for i in range(n):
            x=int(input(("Enter value:")))
            new=Node(x)
            if self.head is None:
                self.head=new
            else:
                temp=self.head
                while temp.next:
                    temp=temp.next
                temp.next=new
    def insert_begin(self, data):
        new=Node(data)
        new.next=self.head
        self.head=new
    def insert_end(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
        else:
            temp=self.head
            while temp.next:
                temp=temp.next
            temp.next=new

    def insert_index(self,index,data):
        if index==0:
            self.insert_begin(data)
            return
        new=Node(data)
        temp=self.head
        for i in range(index-1):
            if temp is None:
                print("Invalid index")
                return
            temp=temp.next
            new.next=temp.next
```

SLL.py - C:\Users\visr\Desktop\DSA Lab\SLLpy (3.13.7)

File Edit Format Run Options Window Help

```
            new.next=temp.next
            temp.next=new

    def deleteAtbeg(self):
        if self.head is None:
            print("No data to delete")
        else:
            temp=self.head
            self.head=temp.next
            print("Deleted value=",temp.data)

    def deleteAtEnd(self):
        if self.head is None:
            print("No data to delete")
        elif self.head.next is None:
            self.head=None
        else:
            temp=self.head
            temp1=temp
            while temp.next:
                temp1=temp
                temp=temp.next
            temp1.next=None
#delete specific value
    def delete(self,value):
        if self.head is None:
            print("No data to delete")
        else:
            temp=self.head
            if temp and temp.data==value:
                self.head=temp.next
                print("value deleted")
                return
            while temp.next and temp.next.data!=value:
                temp=temp.next
            if temp.next is None:
                print("value is not present")
            else:
                temp.next=temp.next.next
                print("value deleted")
  #display/traversal
    def display(self):
        if self.head is None:
            print("No data")
        else:
            temp=self.head
            while temp:
```

SLL.py - C:\Users\visr\Desktop\DSA Lab\SLLpy (3.13.7)

File Edit Format Run Options Window Help

```
            temp=self.head
            while temp:
                print(temp.data,end="->")
                temp=temp.next
            print("None")
#count number of nodes
    def count(self):
        if self.head is None:
            print("No Linked List")
        else:
            c=0
            temp=self.head
            while temp:
                c+=1
                temp=temp.next
            print(f"Number of nodes={c}")

sll=SinglyLinkedListList()
while True:

    print("\n1.Create Linked List")
    print("2.Insert Beginning")
    print("3.Insert At End")
    print("4.Insert At Index")
    print("5.Delete At Value")
    print("6.Delete First Node")
    print("7.Delete Last Node")
    print("8.Count Number of Nodes")
    print("9.Display/traverse")
    print("10.Exit")

    ch=int(input("Enter Choice:"))

    if ch==1:
        sll.create()

    if ch==2:
        x=int(input("Enter value:"))
        sll.insert_begin(x)

    elif ch==3:
        x=int(input("Enter value:"))
        sll.insert_end(x)
    elif ch==4:
        idx=int(input("Enter index:"))
        x=int(input("Enter value:"))
        sll.insert_index(idx,x)
```

```
    elif ch==5:
        x=int(input("Enter value to delete:"))
        sll.delete(x)

    elif ch==6:
        sll.deleteAtbeg()

    elif ch==7:
        sll.deleteAtEnd()

    elif ch==8:
        sll.count()

    elif ch==9:
        sll.display()

    elif ch==10:
        break
```

OUTPUT:



```
*IDLE Shell 3.13.7*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:\Users\manee\OneDrive\Desktop\DSA Lab\SLL.py

1.Create Linked List
2.Insert Beginning
3.Insert At End
4.Insert At Index
5.Delete At Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Display/traverse
10.Exit
Enter Choice:1
Enter number of elements:5
Enter value:10
Enter value:20
Enter value:30
Enter value:40
Enter value:50

1.Create Linked List
2.Insert Beginning
3.Insert At End
4.Insert At Index
5.Delete At Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Display/traverse
10.Exit
Enter Choice:9
10->20->30->40->50->None

1.Create Linked List
2.Insert Beginning
3.Insert At End
4.Insert At Index
5.Delete At Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Display/traverse
10.Exit
Enter Choice:
```

WEEK-6

AIM:

Write a python program to implement a Double Linked List with the below operations.

1) Create Linked List
2) Insert Beginning
3) Insert End
4) Insert at Index
5) Delete by Value
6) Delete First Node
7) Delete Last Node
8) Count number of nodes
9) Display or Traverse
10) Exit

CODE:

DLL.py - C:\Users\visr\Desktop\DSA Lab\DLLpy (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    # Create Linked List
    def create(self):
        n = int(input("Enter number of elements: "))
        for i in range(n):
            x = int(input("Enter value: "))
            self.insert_end(x)

    # Insert at Beginning
    def insert_begin(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
            self.tail = new
        else:
            new.next = self.head
            self.head.prev = new
            self.head = new

    # Insert at End
    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
            self.tail = new
        else:
            self.tail.next = new
            new.prev = self.tail
            self.tail = new

    # Insert at Index
    def insert_index(self, index, data):
        if index < 0 or index > self.count():
            print("Invalid Index")
```

DLL.py - C:\Users\visr\Desktop\DSA Lab\DLL.py (3.13.7)

File Edit Format Run Options Window Help

```
        if index < 0 or index > self.count():
            print("Invalid Index")
            return

        if index == 0:
            self.insert_begin(data)
            return

        if index == self.count():
            self.insert_end(data)
            return

        temp = self.head

        for i in range(index - 1):
            temp = temp.next

        new = Node(data)
        new.next = temp.next
        new.prev = temp
        temp.next.prev = new
        temp.next = new

        print("Insertion Completed")

    # Delete by Value
    def delete(self, value):
        temp = self.head

        while temp and temp.data != value:
            temp = temp.next

        if temp is None:
            print("Value not found")
            return

        if temp.prev:
            temp.prev.next = temp.next
        else:
            self.head = temp.next

        if temp.next:
            temp.next.prev = temp.prev
        else:
            self.tail = temp.prev

        print("Value Deleted")
```

DLL.py - C:\Users\visr\Desktop\DSA Lab\DLL.py (3.13.7)

File Edit Format Run Options Window Help

```
        print("Value Deleted")

    # Delete First Node
    def delete_begin(self):
        if self.head is None:
            print("Can't perform delete operation")
        elif self.head.next is None:
            self.head = None
            self.tail = None
            print("Value Deleted")
        else:
            self.head = self.head.next
            self.head.prev = None
            print("Value Deleted")

    # Delete Last Node
    def delete_end(self):
        if self.tail is None:
            print("Can't perform delete operation")
        elif self.head == self.tail:
            self.head = None
            self.tail = None
            print("Value Deleted")
        else:
            self.tail = self.tail.prev
            self.tail.next = None
            print("Value Deleted")

    # Count Number of Nodes
    def count(self):
        c = 0
        temp = self.head

        while temp:
            c += 1
            temp = temp.next

        return c

    # Display Forward
    def display_forward(self):
        temp = self.head

        if temp is None:
            print("No data")
            return
```

DLL.py - C:\Users\visr\Desktop\DSA Lab\DLL.py (3.13.7)

File Edit Format Run Options Window Help

```
            print("No data")
            return

        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")

    # Display Backward
    def display_backward(self):
        temp = self.tail

        if temp is None:
            print("No data")
            return

        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.prev

        print("None")


dll = DoublyLinkedList()

while True:
    print("\n1.Create Linked List")
    print("2.Insert Beginning")
    print("3.Insert End")
    print("4.Insert at Index")
    print("5.Delete by Value")
    print("6.Delete First Node")
    print("7.Delete Last Node")
    print("8.Count Number of Nodes")
    print("9.Display / Traverse")
    print("10.Exit")

    ch = int(input("Enter Choice: "))

    if ch == 1:
        dll.create()

    elif ch == 2:
        x = int(input("Enter value: "))
        dll.insert_begin(x)
```

```
    elif ch == 3:
        x = int(input("Enter value: "))
        dll.insert_end(x)

    elif ch == 4:
        idx = int(input("Enter index: "))
        x = int(input("Enter value: "))
        dll.insert_index(idx, x)

    elif ch == 5:
        x = int(input("Enter value to delete: "))
        dll.delete(x)

    elif ch == 6:
        dll.delete_begin()

    elif ch == 7:
        dll.delete_end()

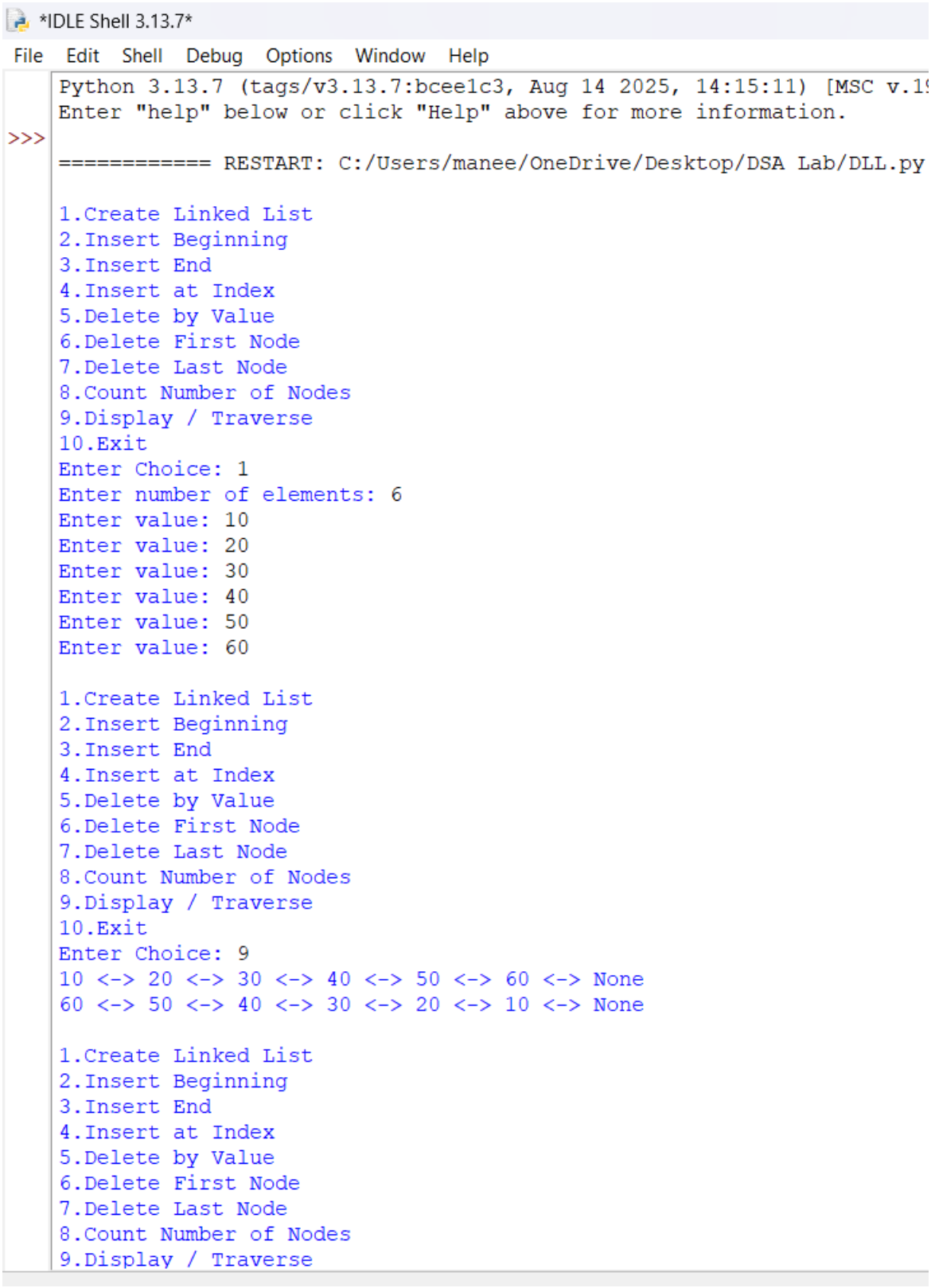
    elif ch == 8:
        print("Number of nodes =", dll.count())

    elif ch == 9:
        dll.display_forward()
        dll.display_backward()

    elif ch == 10:
        break

    else:
        print("Invalid choice")
```

OUTPUT:



```
*IDLE Shell 3.13.7*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.19
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/DLL.py

1.Create Linked List
2.Insert Beginning
3.Insert End
4.Insert at Index
5.Delete by Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Display / Traverse
10.Exit
Enter Choice: 1
Enter number of elements: 6
Enter value: 10
Enter value: 20
Enter value: 30
Enter value: 40
Enter value: 50
Enter value: 60

1.Create Linked List
2.Insert Beginning
3.Insert End
4.Insert at Index
5.Delete by Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Display / Traverse
10.Exit
Enter Choice: 9
10 <-> 20 <-> 30 <-> 40 <-> 50 <-> 60 <-> None
60 <-> 50 <-> 40 <-> 30 <-> 20 <-> 10 <-> None

1.Create Linked List
2.Insert Beginning
3.Insert End
4.Insert at Index
5.Delete by Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Display / Traverse
```

WEEK-7

AIM:

7(A). Write a python program to implement stack operations such as push, pop, peek, display using an array.

CODE:

Stack_Operations_Using_Arrays.py - C:\Users\visr\Desktop\DSA Lab\Stack_Operations_Using_Arrays.py

File Edit Format Run Options Window Help

```
class StackEx:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    # Push operation
    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    # Pop operation
    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")

    # Peek operation
    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    # Display operation
    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack are:")
            for i in range(self.top, -1, -1):
                print(self.stack[i])


size = int(input("Enter stack size: "))
s = StackEx(size)
```

```
while True:
    print("\n1.Push")
    print("2.Pop")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        item = int(input("Enter element: "))
        s.push(item)

    elif ch == 2:
        s.pop()

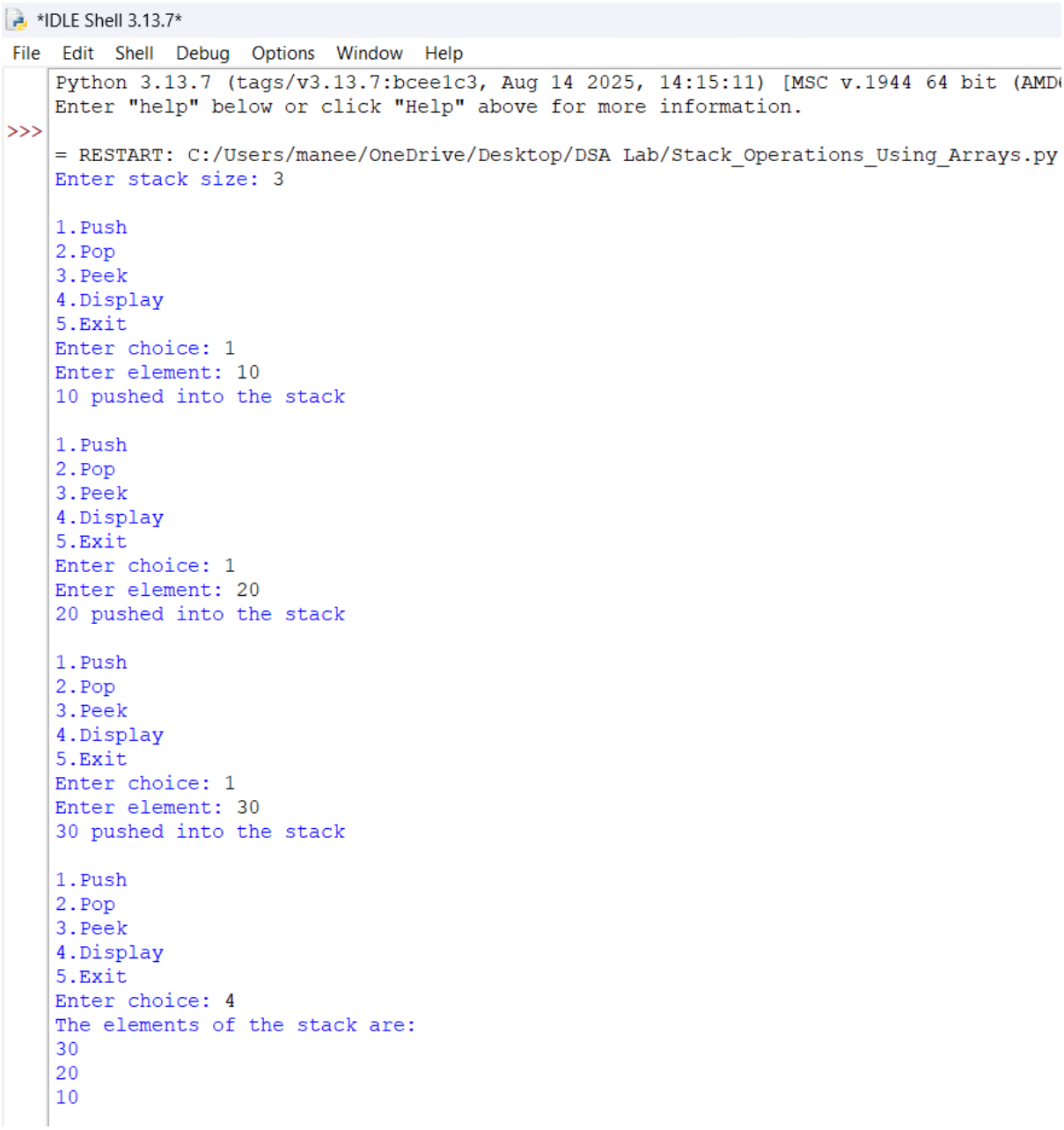
    elif ch == 3:
        s.peek()

    elif ch == 4:
        s.display()

    elif ch == 5:
        break

    else:
        print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Stack_Operations_Using_Arrays.py
Enter stack size: 3
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter element: 10
10 pushed into the stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter element: 20
20 pushed into the stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter element: 30
30 pushed into the stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 4
The elements of the stack are:
30
20
10

```
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 2
30 popped from the stack

1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 4
The elements of the stack are:
20
10
```

7(B). Write a python program to implement stack operations such as push, pop, peek, display using a linked list

CODE:

Stack_Operations_Using_SLL.py - C:\Users\visr\Desktop\DSA Lab\Stack_Operations_Using_SLL.py

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    # Push operation
    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new
        print(data, "pushed into stack")

    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp = self.top
            self.top = self.top.next
            print(temp.data, "popped from stack")

    # Peek operation
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    # Display operation
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            while temp:
                print(temp.data, end=" ")
                temp = temp.next
            print()

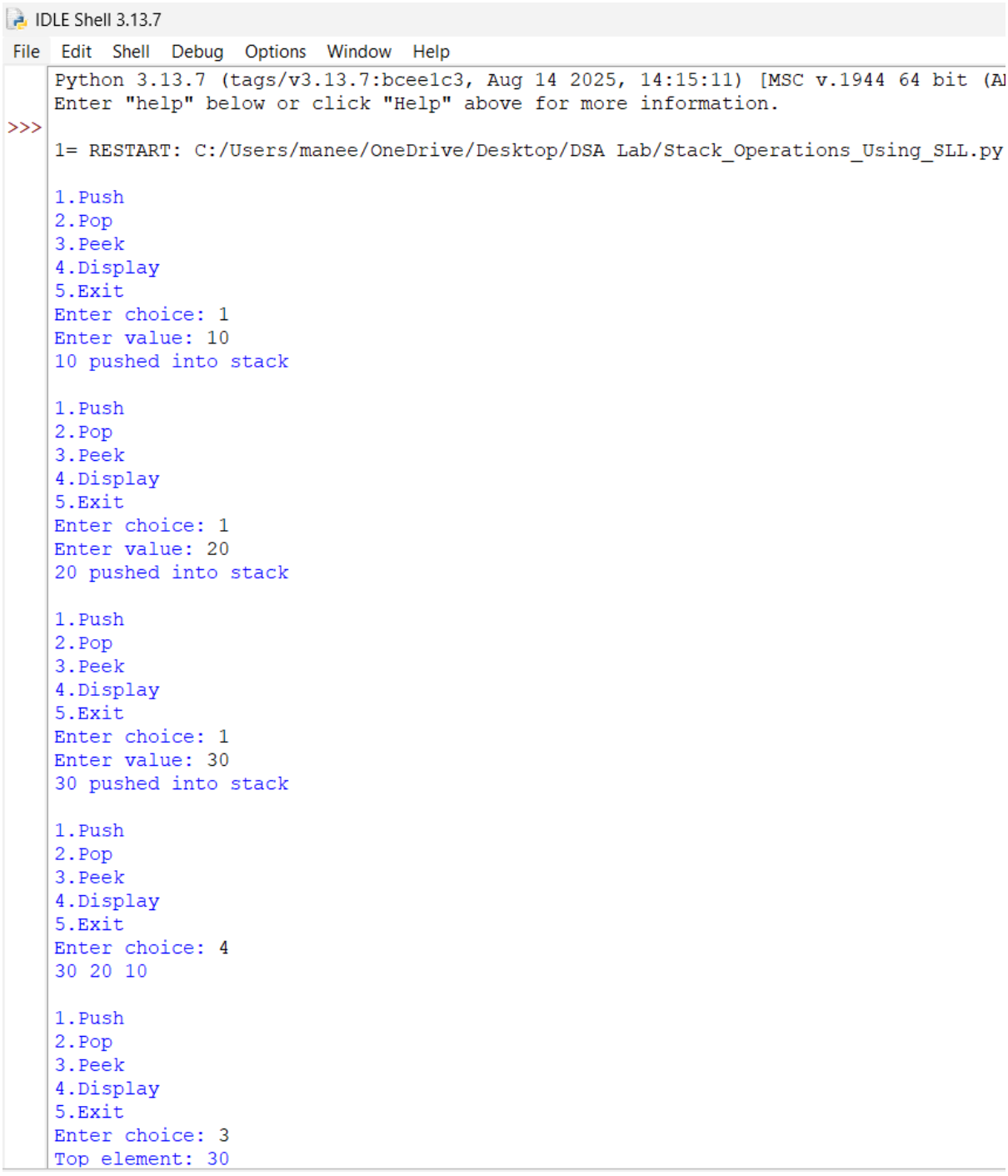
s = Stack()
```

```
while True:
    print("\n1.Push")
    print("2.Pop")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        data = int(input("Enter value: "))
        s.push(data)
    elif ch == 2:
        s.pop()
    elif ch == 3:
        s.peek()
    elif ch == 4:
        s.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (A
Enter "help" below or click "Help" above for more information.
>>>
1= RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Stack_Operations_Using_SLL.py
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter value: 10
10 pushed into stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter value: 20
20 pushed into stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter value: 30
30 pushed into stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 4
30 20 10
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 3
Top element: 30

```
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 2
30 popped from stack

1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 4
20 10

1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 5
```

WEEK-8

AIM:

8(A). Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

CODE:

Using Array:

Queue_Using_Array.py - CC:\Users\visr\Desktop\DSA Lab\Queue_Using_Array.py (3.13.7)

File Edit Format Run Options Window Help

```
class QueueEx:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    # Enqueue operation
    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
            self.queue[self.rear] = x
            print(x, "inserted into the queue")

    # Dequeue operation
    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(x, "deleted from the queue")

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    # Peek operation
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
```

```
            for i in range(self.front, self.rear + 1):
                print(self.queue[i], end=" ")
            print()

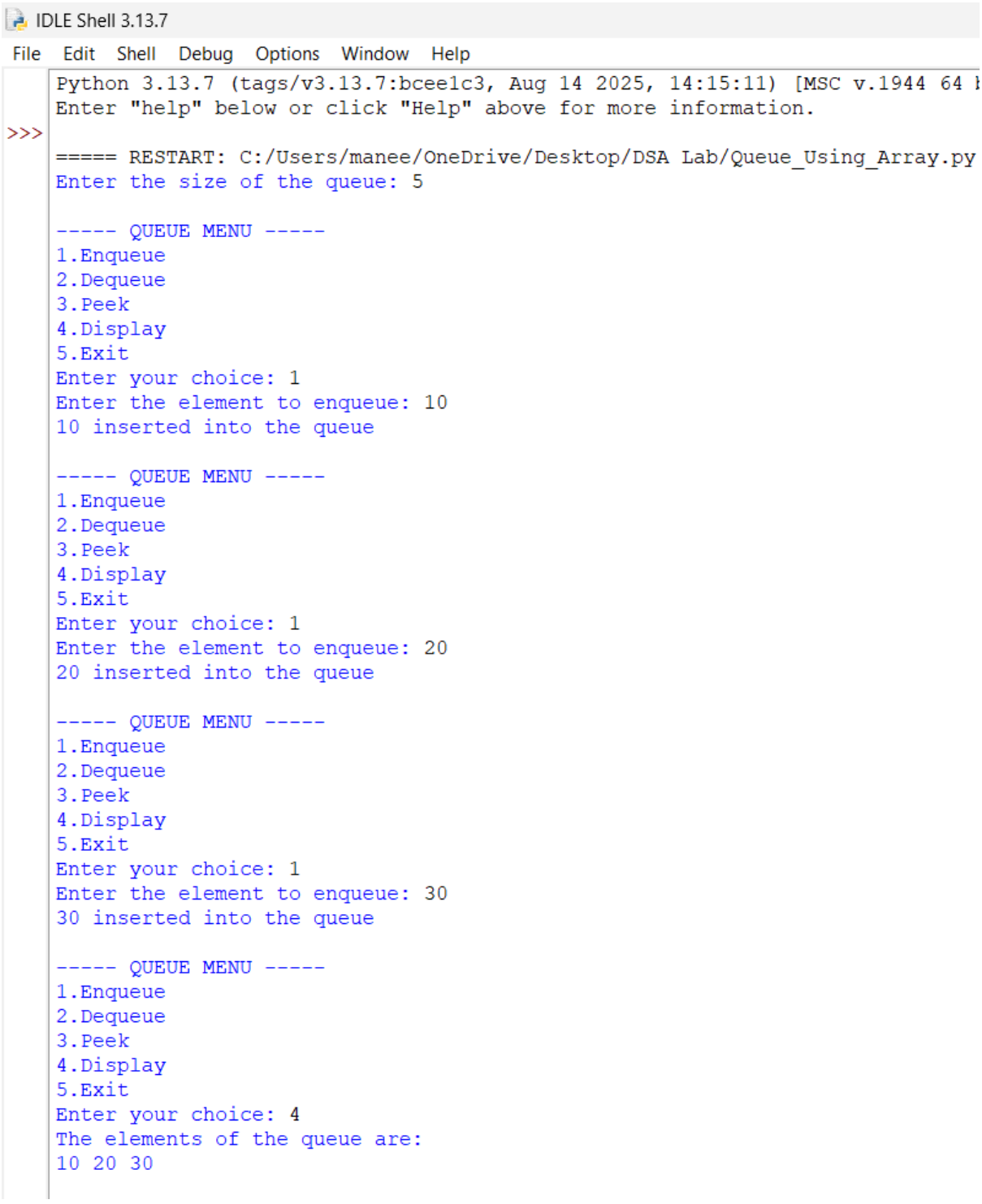
size = int(input("Enter the size of the queue: "))
q = QueueEx(size)

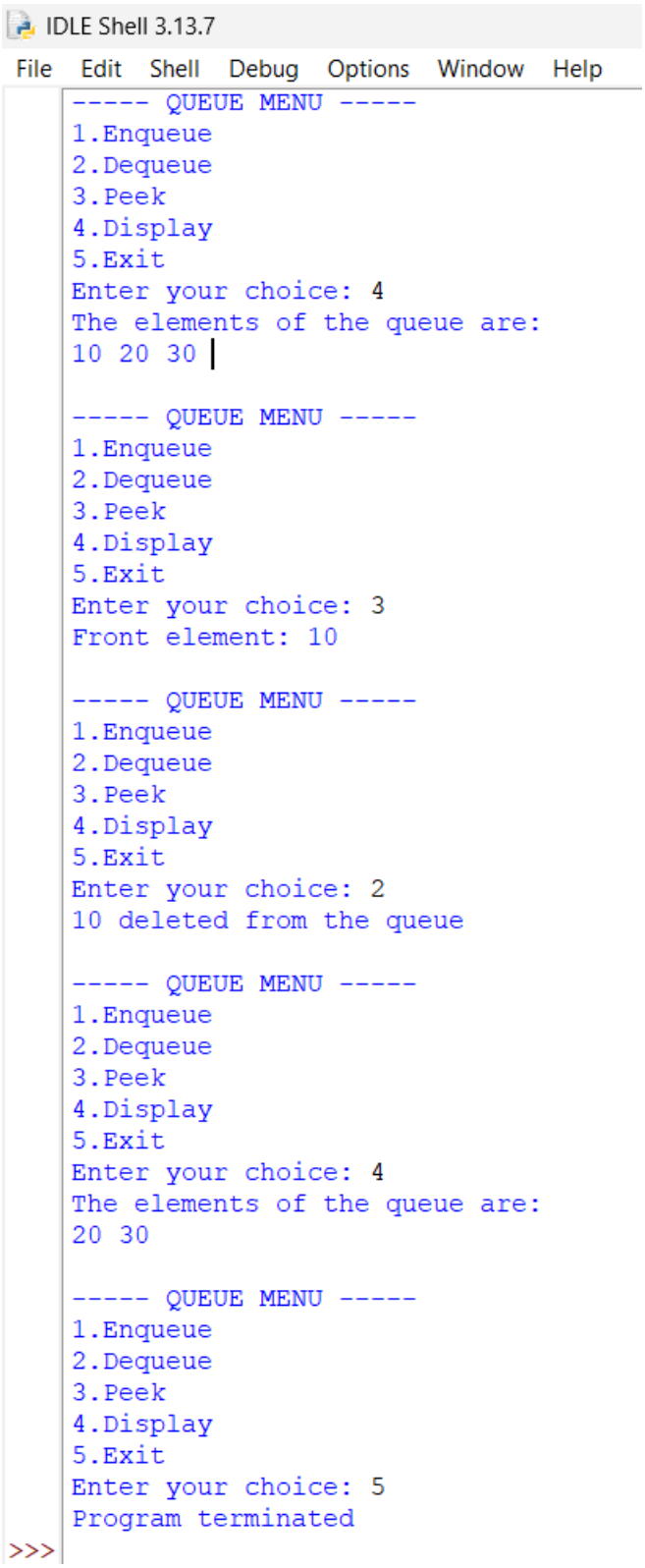
while True:
    print("\n----- QUEUE MENU -----")
    print("1.Enqueue")
    print("2.Dequeue")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 b
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Queue_Using_Array.py
Enter the size of the queue: 5
----- QUEUE MENU -----
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter the element to enqueue: 10
10 inserted into the queue
----- QUEUE MENU -----
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter the element to enqueue: 20
20 inserted into the queue
----- QUEUE MENU -----
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter the element to enqueue: 30
30 inserted into the queue
----- QUEUE MENU -----
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 4
The elements of the queue are:
10 20 30


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
----- QUEUE MENU -----
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 4
The elements of the queue are:
10 20 30
----- QUEUE MENU -----
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 3
Front element: 10
----- QUEUE MENU -----
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 2
10 deleted from the queue
----- QUEUE MENU -----
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 4
The elements of the queue are:
20 30
----- QUEUE MENU -----
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 5
Program terminated
>>>

CODE:

Using SLL:

Queue_Using_SLL.py - C:\Users\visr\Desktop\DSA Lab\Queue_Using_SLLpy

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new = Node(data)

        if self.rear is None:
            self.front = self.rear = new
        else:
            self.rear.next = new
            self.rear = new

        print(data, "inserted into the queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            temp = self.front
            print(temp.data, "deleted from the queue")
            self.front = self.front.next

            if self.front is None:
                self.rear = None

    def peek(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            print("Front element:", self.front.data)

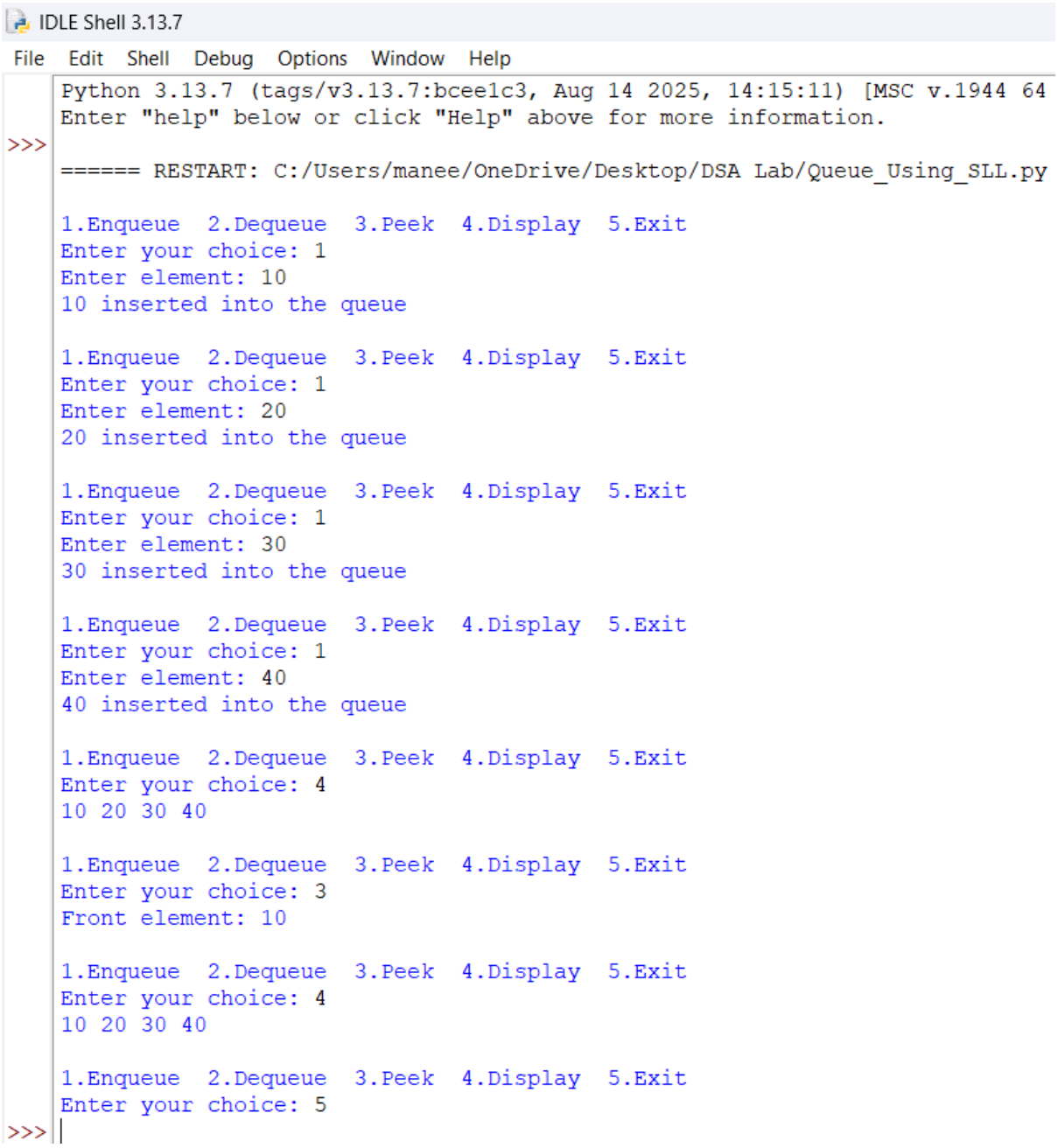
    def display(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            temp = self.front
            while temp:
                print(temp.data, end=" ")
                temp = temp.next
            print()
```

```
q = Queue()

while True:
    print("\n1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit")
    ch = int(input("Enter your choice: "))

    if ch == 1:
        x = int(input("Enter element: "))
        q.enqueue(x)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
    elif ch == 4:
        q.display()
    elif ch == 5:
        break
    else:
        print("Invalid Choice")
```

OUTPUT:



8(B). Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

CODE:

Using Array:

Circular_Queue_Using_Array.py - C:\Users\visr\Desktop\DSA Lab\Circular_Queue_Using_Array.py

File Edit Format Run Options Window Help

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None

            if self.front == self.rear:
                self.front = -1
                self.rear = -1
            else:
                self.front = (self.front + 1) % self.size

            print(item, "deleted from queue")

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            i = self.front
            while True:
```

```
            while True:
                print(self.queue[i], end=" ")
                if i == self.rear:
                    break
                i = (i + 1) % self.size
            print()

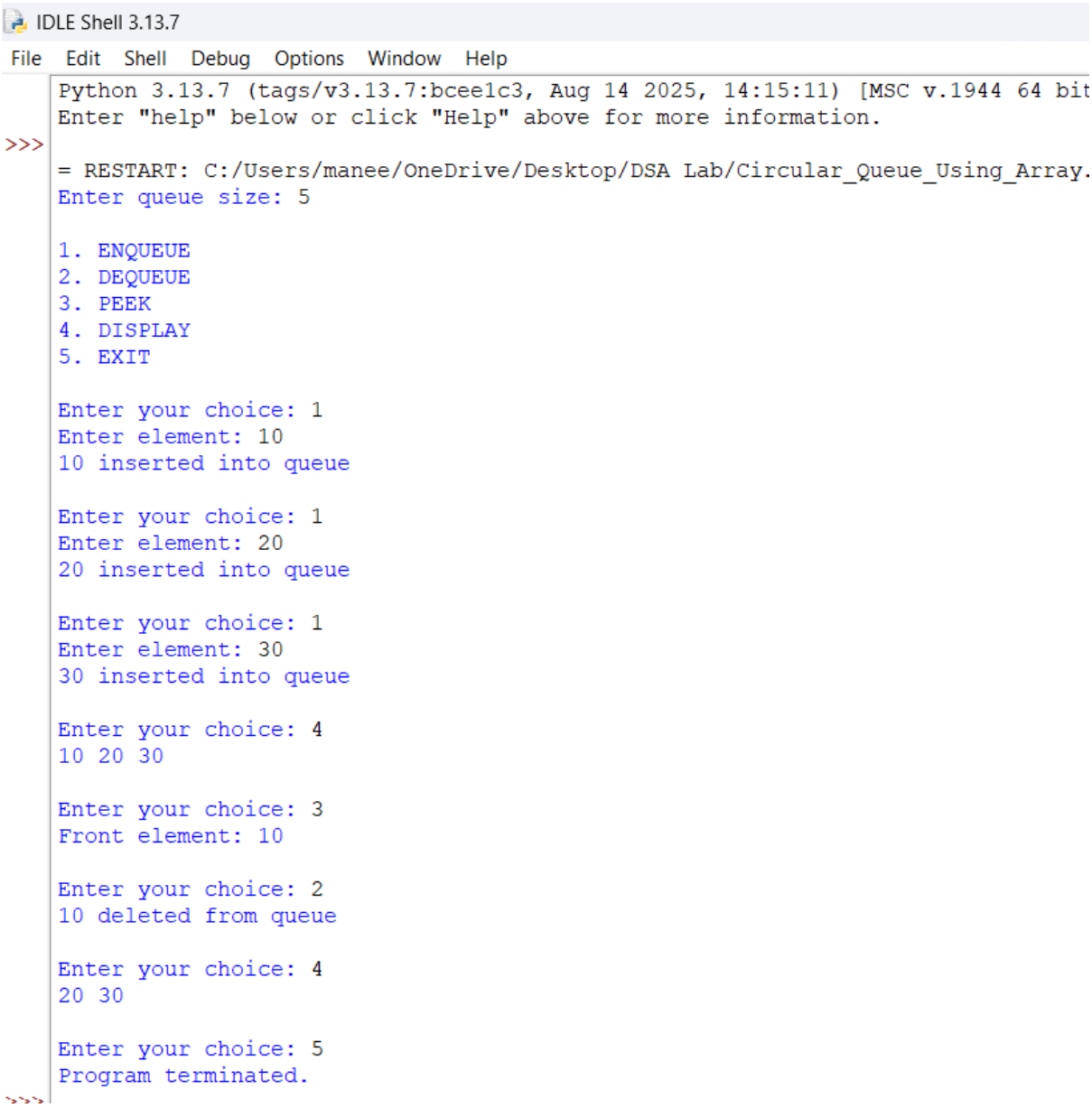
size = int(input("Enter queue size: "))
q = CircularQueue(size)

print("\n1. ENQUEUE")
print("2. DEQUEUE")
print("3. PEEK")
print("4. DISPLAY")
print("5. EXIT")

while True:
    ch = int(input("\nEnter your choice: "))

    if ch == 1:
        item = int(input("Enter element: "))
        q.enqueue(item)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
    elif ch == 4:
        q.display()
    elif ch == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Circular_Queue_Using_Array.
Enter queue size: 5
1. ENQUEUE
2. DEQUEUE
3. PEEK
4. DISPLAY
5. EXIT
Enter your choice: 1
Enter element: 10
10 inserted into queue
Enter your choice: 1
Enter element: 20
20 inserted into queue
Enter your choice: 1
Enter element: 30
30 inserted into queue
Enter your choice: 4
10 20 30
Enter your choice: 3
Front element: 10
Enter your choice: 2
10 deleted from queue
Enter your choice: 4
20 30
Enter your choice: 5
Program terminated.
>>>

CODE:

Using SLL:

Circular_Queue_Using_SLL.py - C:\Users\visr\Desktop\DSA Lab\Circular_Queue Using SLL py

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, item):
        new = Node(item)

        if self.front is None:
            self.front = self.rear = new
            self.rear.next = self.front
        else:
            self.rear.next = new
            self.rear = new
            self.rear.next = self.front

        print(item, "inserted into queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            item = self.front.data

            if self.front == self.rear:
                self.front = None
                self.rear = None
            else:
                self.front = self.front.next
                self.rear.next = self.front

            print(item, "deleted from queue")

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
```

```
    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            temp = self.front
            while True:
                print(temp.data, end=" ")
                temp = temp.next
                if temp == self.front:
                    break
            print()

q = CircularQueue()

print("\n1. ENQUEUE")
print("2. DEQUEUE")
print("3. PEEK")
print("4. DISPLAY")
print("5. EXIT")

while True:
    ch = int(input("\nEnter your choice: "))

    if ch == 1:
        item = int(input("Enter element: "))
        q.enqueue(item)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
    elif ch == 4:
        q.display()
    elif ch == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
>>>
= RESTAR: C:\Users\visr\Desktop\DSA Lab\Circular_Queue_Using_SLL.py

1. ENQUEUE
2. DEQUEUE
3. PEEK
4. DISPLAY
5. EXIT

Enter your choice: 1
Enter element: 10
10 inserted into queue

Enter your choice: 1
Enter element: 20
20 inserted into queue

Enter your choice: 1
Enter element: 30
30 inserted into queue

Enter your choice: 1
Enter element: 40
40 inserted into queue

Enter your choice: 4
10 20 30 40

Enter your choice: 3
Front element: 10

Enter your choice: 2
10 deleted from queue

Enter your choice: 4
20 30 40

Enter your choice: 5
Program terminated.
>>>
```

WEEK-9:

AIM:

Write a python program to implement a Binary Search Tree (BST) and perform the following operations.

1. Insertion
2. Deletion
3. Tree Traversals – in-order, pre-order and post-order
4. Count Leaf Nodes
5. Count Internal Nodes
6. Height of the Tree

CODE:

BST.py - C:\Users\visr\Desktop\DSA Lab\BST.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")

def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
```

BST.py - C:\Users\visr\Desktop\DSA Lab\BST.py (3.13.7)

File Edit Format Run Options Window Help

```
def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1


def delete(root, x):
    if root is None:
```

BST.py - C:\Users\visr\Desktop\DSA Lab\BST.py (3.13.7)

File Edit Format Run Options Window Help

```
def delete(root, x):
    if root is None:
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
            root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)


def level_order(root):
    h = height(root)
    for level in range(h):
        print_level(root, level)


# ======================================================
# MAIN PROGRAM
# ======================================================

root = None
while True:
```

BST.py - C:\Users\visr\Desktop\DSA Lab\BST.py (3.13.7)

File Edit Format Run Options Window Help

```
root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
    print("6. FIND MAXIMUM")
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
    print("10. COUNT LEAVES")
    print("11. COUNT INTERNAL NODES")
    print("12. HEIGHT")
    print("13. LEVEL ORDER TRAVERSAL")
    print("14. EXIT")

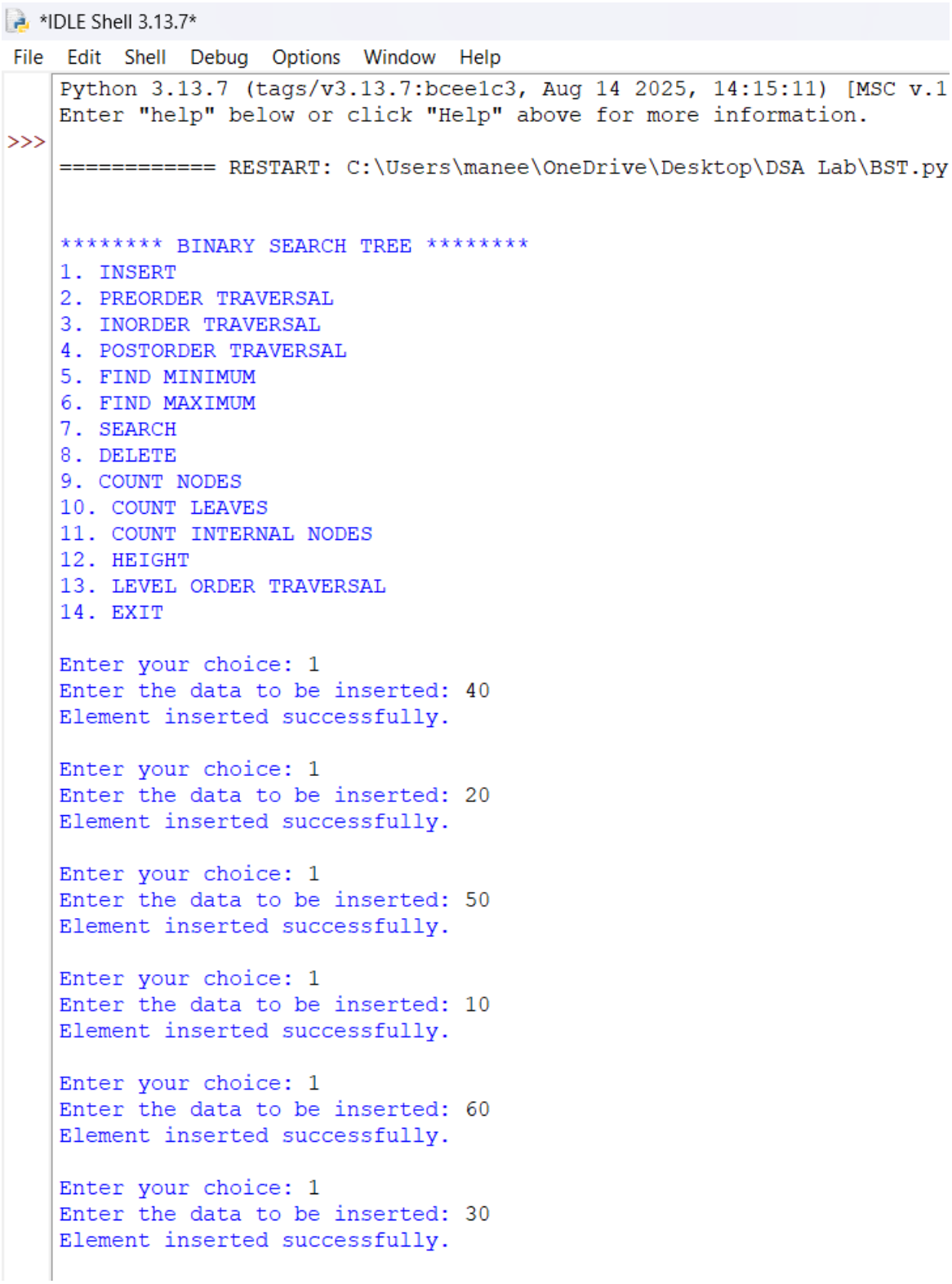
    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            preorder(root)
            print()
    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            inorder(root)
            print()
    elif choice == 4:
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            postorder(root)
            print()
    elif choice == 5:
        node = find_min(root)
        if node is None:
```

BST.py - C:\Users\visr\Desktop\DSA Lab\BST.py (3.13.7)

File Edit Format Run Options Window Help

```
            print("Tree is empty.")
        else:
            postorder(root)
            print()
    elif choice == 5:
        node = find_min(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")
    elif choice == 8:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 9:
        print("Number of nodes:", count_nodes(root))
    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 12:
        print("Height of BST:", height(root))
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()
    elif choice == 14:
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:\Users\manee\OneDrive\Desktop\DSA Lab\BST.py
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 40
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 20
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 50
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 10
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 60
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 30
Element inserted successfully.

```
Enter your choice: 2
Preorder Traversal: 40 20 10 30 50 60

Enter your choice: 3
Inorder Traversal: 10 20 30 40 50 60

Enter your choice: 4
Postorder Traversal: 10 30 20 60 50 40

Enter your choice: 5
Minimum element: 10

Enter your choice: 6
Maximum element: 60

Enter your choice: 7
Enter the data to be searched: 30
Element 30 is found.

Enter your choice: 9
Number of nodes: 6

Enter your choice: 10
Number of leaf nodes: 3

Enter your choice: 11
Number of internal nodes: 3

Enter your choice: 12
Height of BST: 2

Enter your choice: 12
Height of BST: 2

Enter your choice: 13
Level Order Traversal: 40 20 50 10 30 60
```

WEEK-10

AIM:

Write a menu driven python program to implement an AVL Tree and perform the following operations.

1. Insertion
2. Deletion
3. Tree Traversals
4. Count Leaf Nodes
5. Count Internal Nodes
6. Height of the Tree

CODE:

AVL_trees.py - C:\Users\visr\Desktop\DSA Lab\AVL_trees.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, x):
        self.left = None
        self.data = x
        self.right = None
        self.h = 0

#AVL TREE Program
def height(root):
    if root is None:
        return -1
    else:
        return root.h

# LL Rotation
def LL(x):
    y = x.left
    x.left = y.right
    y.right = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# RR Rotation
def RR(x):
    y = x.right
    x.right = y.left
    y.left = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# LR Rotation
def LR(k3):
    k3.left = RR(k3.left)
    return LL(k3)

# RL Rotation
def RL(k3):
    k3.right = LL(k3.right)
    return RR(k3)

# Inorder Traversal
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
```

AVL_trees.py - C:\Users\visr\Desktop\DSA Lab\AVL_trees.py (3.13.7)

File Edit Format Run Options Window Help

```
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


# Preorder Traversal
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)


# Postorder Traversal
def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

# Insert a node into AVL Tree
def insert(root, x):
    if root is None:
        return Node(x)

    if x < root.data:
        root.left = insert(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    elif x > root.data:
        root.right = insert(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
    else:
        print("Duplicates not allowed")
    root.h = max(height(root.left),height(root.right)) + 1
    return root


# Delete a node from AVL Tree
```

AVL_trees.py - C:\Users\visr\Desktop\DSA Lab\AVL_trees.py (3.13.7)

File Edit Format Run Options Window Help

```
# Delete a node from AVL Tree
def delete(root, x):
    # Element not found
    if root is None:
        return None

    # Search in left subtree
    if x < root.data:
        root.left = delete(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    # Search in right subtree
    elif x > root.data:
        root.right = delete(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)

    # When target found
    else:
        if root.left is None and root.right is None:
            root = None
        elif root.right is None:
            root = root.left
        elif root.left is None:
            root = root.right
        else:
            successor = root.right   # Find inorder successor
            while successor.left is not None:
                successor = successor.left
            root.data = successor.data   #Copy successor data
            root.right = delete(root.right, successor.data)     # Delete the inorder successor

        root.h = max(height(root.left), height(root.right)) + 1
    return root

# --------------------------------
# Main Program
# --------------------------------
```

AVL_trees.py - C:\Users\visr\Desktop\DSA Lab\AVL_trees.py (3.13.7)

File Edit Format Run Options Window Help

```
            root.data = successor.data    #Copy successor data
            root.right = delete(root.right, successor.data)     # Delete the inorder successor

        root.h = max(height(root.left), height(root.right)) + 1
    return root

# --------------------------------
# Main Program
# --------------------------------

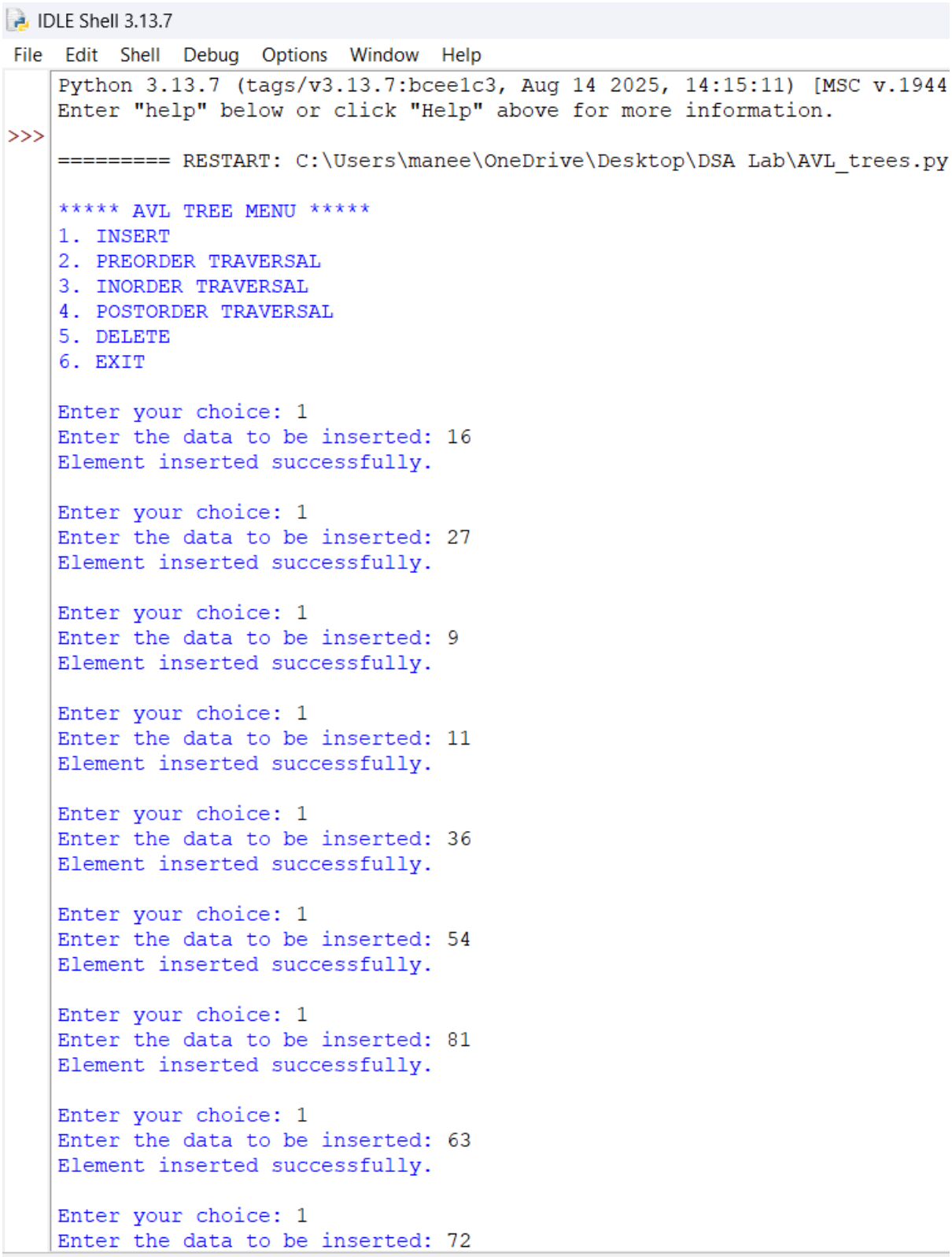
root = None

while True:

    print("\n***** AVL TREE MENU *****")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. DELETE")
    print("6. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
    elif choice == 2:
        print("Preorder Traversal:")
        preorder(root)
        print()
    elif choice == 3:
        print("Inorder Traversal:")
        inorder(root)
        print()
    elif choice == 4:
        print("Postorder Traversal:")
        postorder(root)
        print()
    elif choice == 5:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 6:
        print("Program terminated.")
        break
    else:
        print("Invalid choice!")
```

OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944
Enter "help" below or click "Help" above for more information.
>>>
========= RESTART: C:\Users\manee\OneDrive\Desktop\DSA Lab\AVL_trees.py
***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 1
Enter the data to be inserted: 16
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 27
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 9
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 11
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 36
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 54
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 81
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 63
Element inserted successfully.
Enter your choice: 1
Enter the data to be inserted: 72

```
Enter your choice: 1
Enter the data to be inserted: 14
Element inserted successfully.

Enter your choice: 1
Enter the data to be inserted: 6
Element inserted successfully.

Enter your choice: 2
Preorder Traversal: 16 11 9 6 14 63 36 27 54 81 72

Enter your choice: 3
Inorder Traversal: 6 9 11 14 16 27 36 54 63 72 81

Enter your choice: 4
Postorder Traversal: 6 9 14 11 27 54 36 72 81 63 16

Enter your choice: 6
Program terminated.
```

WEEK-11:

AIM:

Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations:

1. Insertion of an element into the Heap.
2. Deletion of the root element from the Heap.
3. Display the array representation of the Heap.
4. Exit the program.

CODE:

Binary_Heap.py - C:\Users\visr\Desktop\DSA Lab\Binary_Heap.py (3.13.7)

File Edit Format Run Options Window Help

```
def insert(heap, value, heap_type):
    heap.append(value)
    i = len(heap) - 1

    while i > 0:
        parent = (i - 1) // 2

        if (heap_type == "max" and heap[i] > heap[parent]) or \
           (heap_type == "min" and heap[i] < heap[parent]):
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break

def delete(heap, heap_type):
    if len(heap) == 0:
        print("Heap is empty")
        return

    print("Deleted element:", heap[0])
    heap[0] = heap[-1]
    heap.pop()

    n = len(heap)
    i = 0

    while True:
        best = i
        left = 2 * i + 1
        right = 2 * i + 2

        if left < n:
            if (heap_type == "max" and heap[left] > heap[best]) or \
               (heap_type == "min" and heap[left] < heap[best]):
                best = left

        if right < n:
            if (heap_type == "max" and heap[right] > heap[best]) or \
               (heap_type == "min" and heap[right] < heap[best]):
                best = right

        if best != i:
            heap[i], heap[best] = heap[best], heap[i]
            i = best
        else:
            break
```

Binary_Heap.py - C:\Users\visr\Desktop\DSA Lab\Binary_Heap.py (3.13.7)

File Edit Format Run Options Window Help

```
            break

def display(heap):
    if len(heap) == 0:
        print("Heap is empty")
    else:
        print("Heap elements:", heap)

# Main Program

heap = []

print("1. MAX HEAP")
print("2. MIN HEAP")

choice = int(input("Select Heap Type: "))

if choice == 1:
    heap_type = "max"
elif choice == 2:
    heap_type = "min"
else:
    print("Invalid choice")
    exit()

print("\n***** BINARY HEAP MENU *****")
print("1. INSERT")
print("2. DELETE ROOT")
print("3. DISPLAY")
print("4. EXIT")

while True:
    choice = int(input("\nEnter your choice: "))

    if choice == 1:
        value = int(input("Enter value to insert: "))
        insert(heap, value, heap_type)
        print("Element inserted successfully.")

    elif choice == 2:
        delete(heap, heap_type)

    elif choice == 3:
        display(heap)

    elif choice == 4:
        print("Program terminated.")
    break

else:
    print("Invalid choice")
```

OUTPUT:

Max Heap:



```
Enter your choice: 3
Heap elements: [72, 54, 63, 27, 36, 45, 61, 18]

Enter your choice: 2
Deleted element: 72

Enter your choice: 3
Heap elements: [63, 54, 61, 27, 36, 45, 18]

Enter your choice: 4
Program terminated.
```

Min Heap:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:/Users/manee/OneDrive/Desktop/DSA Lab/Binary_Heap.py
1. MAX HEAP
2. MIN HEAP
Select Heap Type: 2
***** BINARY HEAP MENU *****
1. INSERT
2. DELETE ROOT
3. DISPLAY
4. EXIT
Enter your choice: 1
Enter value to insert: 28
Element inserted successfully.
Enter your choice: 1
Enter value to insert: 6
Element inserted successfully.
Enter your choice: 1
Enter value to insert: 4
Element inserted successfully.
Enter your choice: 1
Enter value to insert: 16
Element inserted successfully.
Enter your choice: 1
Enter value to insert: 42
Element inserted successfully.
Enter your choice: 1
Enter value to insert: 3
Element inserted successfully.
Enter your choice: 1
Enter value to insert: 12
Element inserted successfully.
Enter your choice: 1
Enter value to insert: 15
Element inserted successfully.

```
Enter your choice: 1
Enter value to insert: 2
Element inserted successfully.

Enter your choice: 3
Heap elements: [2, 3, 4, 15, 42, 6, 12, 28, 16]

Enter your choice: 2
Deleted element: 2

Enter your choice: 3
Heap elements: [3, 15, 4, 16, 42, 6, 12, 28]

Enter your choice: 4
Program terminated.
```

WEEK-12:

AIM:

Write a program to implement the following three collision resolution techniques in a Hash Table:

1. Separate Chaining
2. Linear Probing
3. Quadratic Probing

1. Separate Chaining

CODE:

Hashing_Separate_Chaining.py - C:\Users\visr\Desktop\DSA Lab\Hashing_Separate_Chaining.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

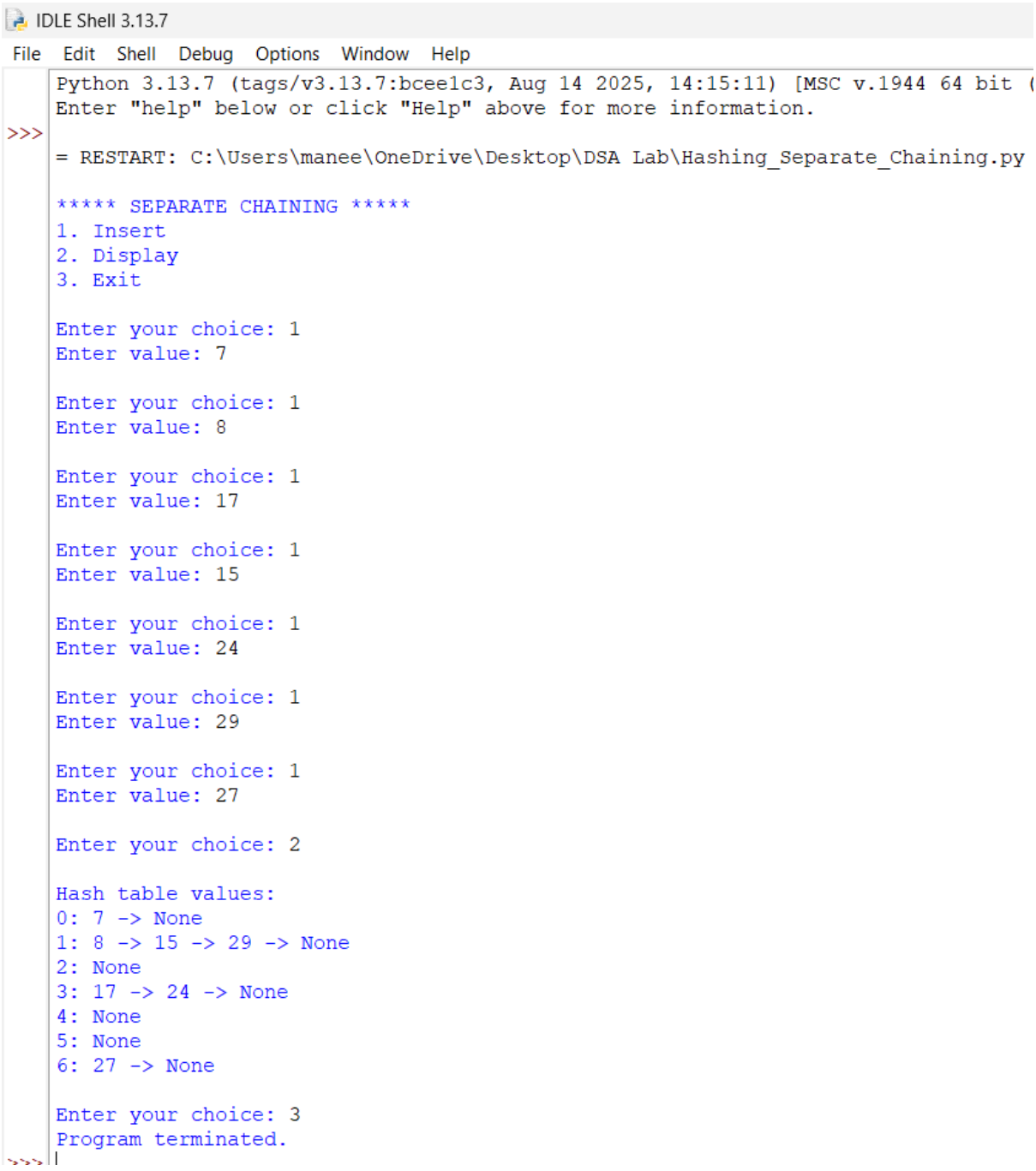
SIZE = 10
ht = [None] * SIZE

def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode

def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("\nHash table values:")
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\manee\OneDrive\Desktop\DSA Lab\Hashing_Separate_Chaining.py
***** SEPARATE CHAINING *****
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 7
Enter your choice: 1
Enter value: 8
Enter your choice: 1
Enter value: 17
Enter your choice: 1
Enter value: 15
Enter your choice: 1
Enter value: 24
Enter your choice: 1
Enter value: 29
Enter your choice: 1
Enter value: 27
Enter your choice: 2
Hash table values:
0: 7 -> None
1: 8 -> 15 -> 29 -> None
2: None
3: 17 -> 24 -> None
4: None
5: None
6: 27 -> None
Enter your choice: 3
Program terminated.
>>>

2.Linear Probing

CODE:

Hashing_Linear_Probing.py - C:\Users\visr\Desktop\DSA Lab\Hashing_Linear_Probing.py (3.13.7)

File Edit Format Run Options Window Help

```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")


def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")

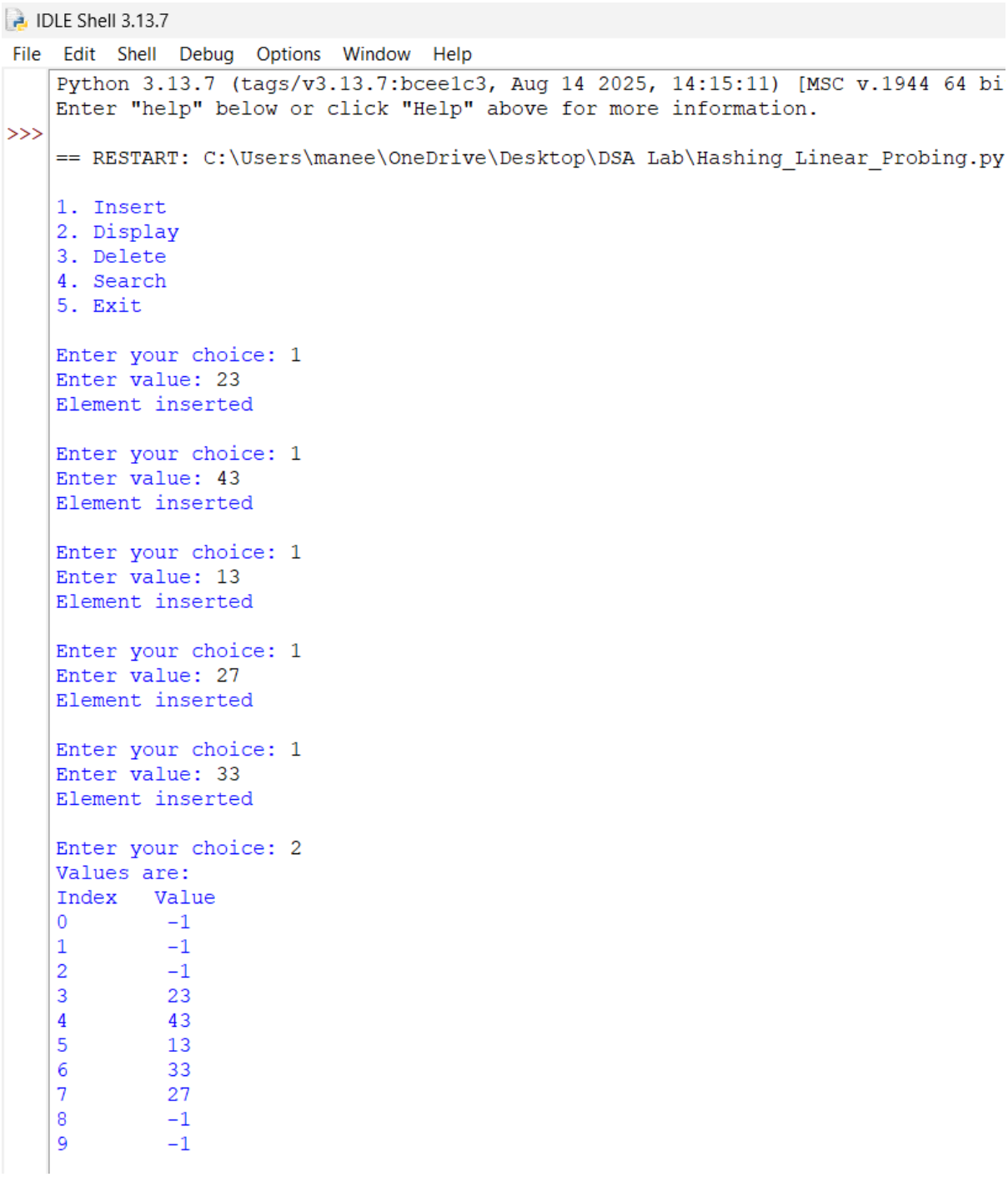
def search(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            print("Element found at index", index)
            return
        if a[index] == -1:
            break
    print("Element not found")


def displayValue():
    print("Index\tValue")
    for i in range(SIZE):
        print(i, "\t", a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Delete")
    print("4. Search")
    print("5. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
```

```
if ch == 1:
    value = int(input("Enter value: "))
    insertValue(value)
elif ch == 2:
    print("Values are:")
    displayValue()
elif ch == 3:
    value = int(input("Enter value to delete: "))
    deleteValue(value)
elif ch == 4:
    value = int(input("Enter value to search: "))
    search(value)
elif ch == 5:
    repeat = False
else:
    print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bi
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:\Users\manee\OneDrive\Desktop\DSA Lab\Hashing_Linear_Probing.py
1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 1
Enter value: 23
Element inserted
Enter your choice: 1
Enter value: 43
Element inserted
Enter your choice: 1
Enter value: 13
Element inserted
Enter your choice: 1
Enter value: 27
Element inserted
Enter your choice: 1
Enter value: 33
Element inserted
Enter your choice: 2
Values are:
Index Value
0 -1
1 -1
2 -1
3 23
4 43
5 13
6 33
7 27
8 -1
9 -1

```
Enter your choice: 4
Enter value to search: 27
Element found at index 7

Enter your choice: 3
Enter value to delete: 43
Element deleted

Enter your choice: 2
Values are:
Index   Value
0        -1
1        -1
2        -1
3        23
4        -1
5        13
6        33
7        27
8        -1
9        -1

Enter your choice: 5
```

3.Quadratic Probing

CODE:

*Hashing_Quadratic_Probing.py - C:\Users\visr\Desktop\DSA Lab\Hashing_Q

File Edit Format Run Options Window Help

```
SIZE = 10
a = [-1] * SIZE

def insertValue(value):
    for i in range(SIZE):
        index = ((value % SIZE) + i * i) % SIZE

        if a[index] == -1:
            a[index] = value
            print("Element inserted at index", index)
            return

    print("No empty slot found in the probe sequence")

def displayValue():
    print("Values are:")
    for i in range(SIZE):
        print(a[i])

# Menu-driven program
print("\n1. Insert")
print("2. Display")
print("3. Exit")

repeat = True
while repeat:
    ch = int(input("\nEnter your choice: "))

    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)

    elif ch == 2:
        displayValue()

    elif ch == 3:
        repeat = False

    else:
        print("Invalid choice")
```

OUTPUT:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\manee\OneDrive\Desktop\DSA Lab\Hashing_Quadratic_Probing.py

1. Insert
2. Display
3. Exit

Enter your choice: 1
Enter value: 14
Element inserted at index 4

Enter your choice: 1
Enter value: 26
Element inserted at index 6

Enter your choice: 1
Enter value: 38
Element inserted at index 8

Enter your choice: 1
Enter value: 54
Element inserted at index 5

Enter your choice: 1
Enter value: 34
Element inserted at index 3

Enter your choice: 1
Enter value: 67
Element inserted at index 7

Enter your choice: 1
Enter value: 96
Element inserted at index 0

Enter your choice: 1
Enter value: 100
Element inserted at index 1
```

```
Enter your choice: 2
Values are:
96
100
-1
34
14
54
26
67
38
-1

Enter your choice: 3
```